**towards**
data science

Publish AI, ML & data-science insights to a global community of data professionals.

Submit an Article

**Stop Reviewing AI With AI**
Discover why AI reviewing its own code misses the same blind spots every single time.
Register Now

LARGE LANGUAGE MODELS

# Hitchhiker's Guide to RAG: From Tiny Files to Tolstoy with OpenAI's API and LangChain

Scaling a simple RAG pipeline from simple notes to full books

Maria Mouschoutzi
July 11, 2025 • 9 min read

Hanna Barakat & Archival Images of AI + AIxDESIGN / https://betterimagesofai.org / https://creativecommons.org/licenses/by/4.0/

In my latest post, I walked you through setting up a very simple RAG pipeline in Python, using OpenAI's API, LangChain, and your local files. In that post, I cover the very basics of creating embeddings from your local files with LangChain, storing them in a vector database with FAISS, making API calls to OpenAI's API, and ultimately generating responses relevant to your files. 🌟

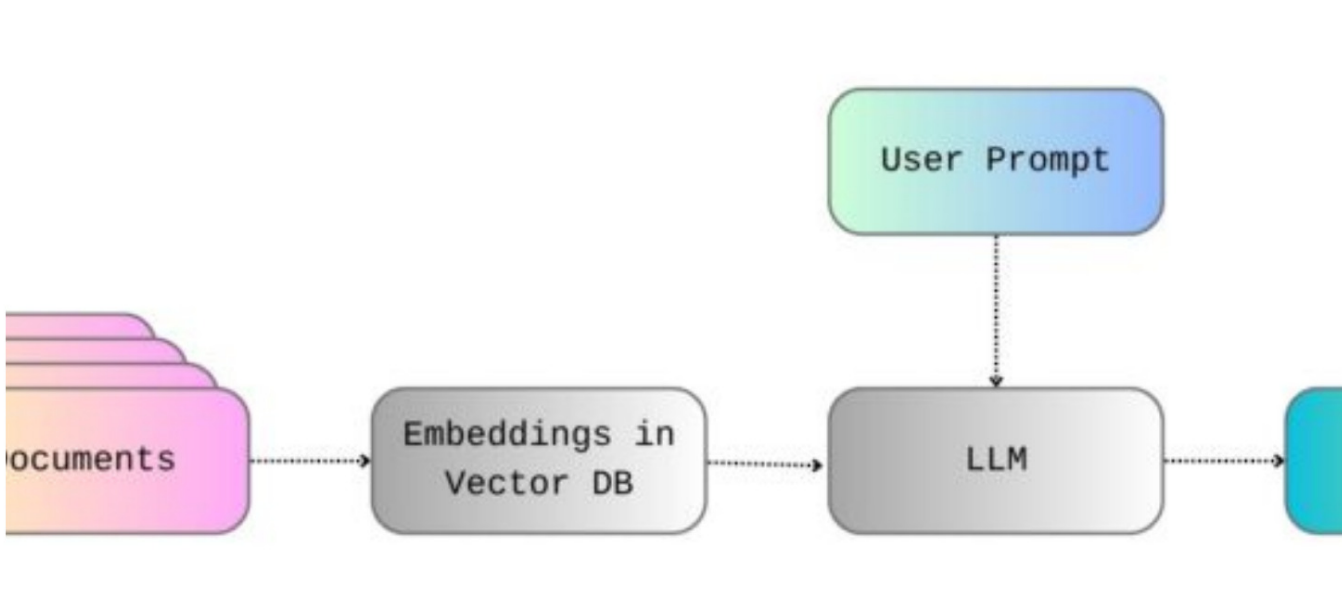


Image by author

Nonetheless, in this simple example, I only demonstrate how to use a tiny .txt file. In this post, I further elaborate on how you can utilize larger files with your RAG pipeline by adding an extra step to the process — ***chunking***.

Learn this step by step with the interactive **AI Engineer** roadmap.

## What about chunking?

Chunking refers to the process of parsing a text into smaller pieces of text—chunks—that are then transformed into embeddings. This is very important because it allows us to effectively process and create embeddings for larger files. All embedding models come with various limitations on the size of the text that is passed — I'll get into more details about those limitations in a moment. These limitations allow for better performance and low-latency responses. In the case that the text we provide doesn't meet those size limitations, it'll get truncated or rejected.

If we wanted to create a RAG pipeline reading, say from Leo Tolstoy's *War and Peace* text (a rather large book), we wouldn't be able to directly load it and transform it into a single embedding. Instead, we need to first do the *chunking* — create smaller chunks of text, and create embeddings for each one. Each chunk being below the size limits of whatever embedding model we use allows us to effectively transform any file into embeddings. So, a *somewhat more* realistic landscape of a RAG pipeline would look as follows:

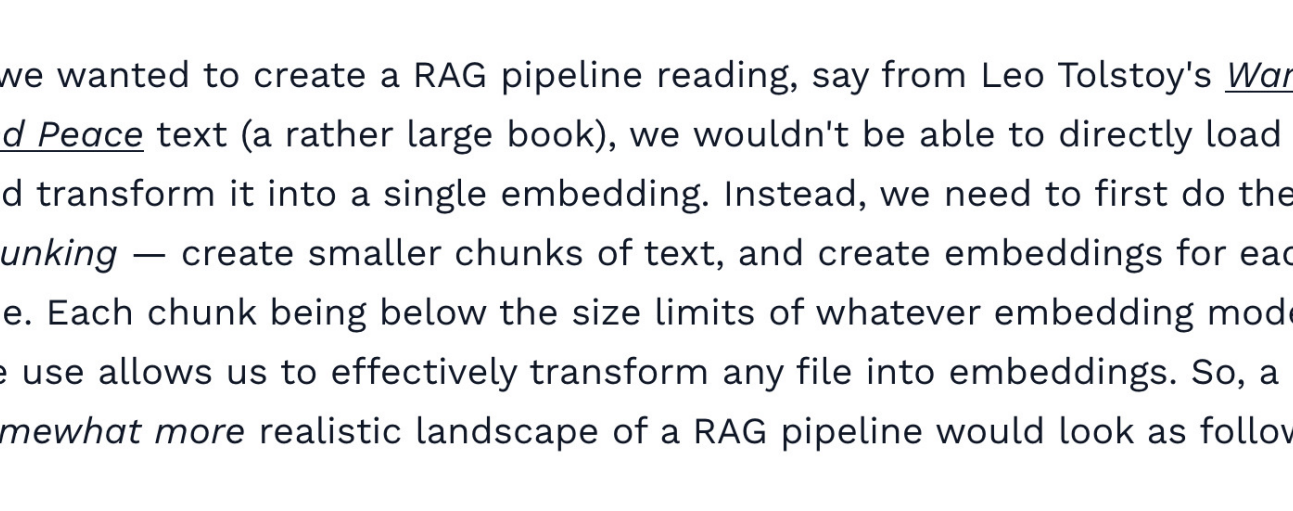
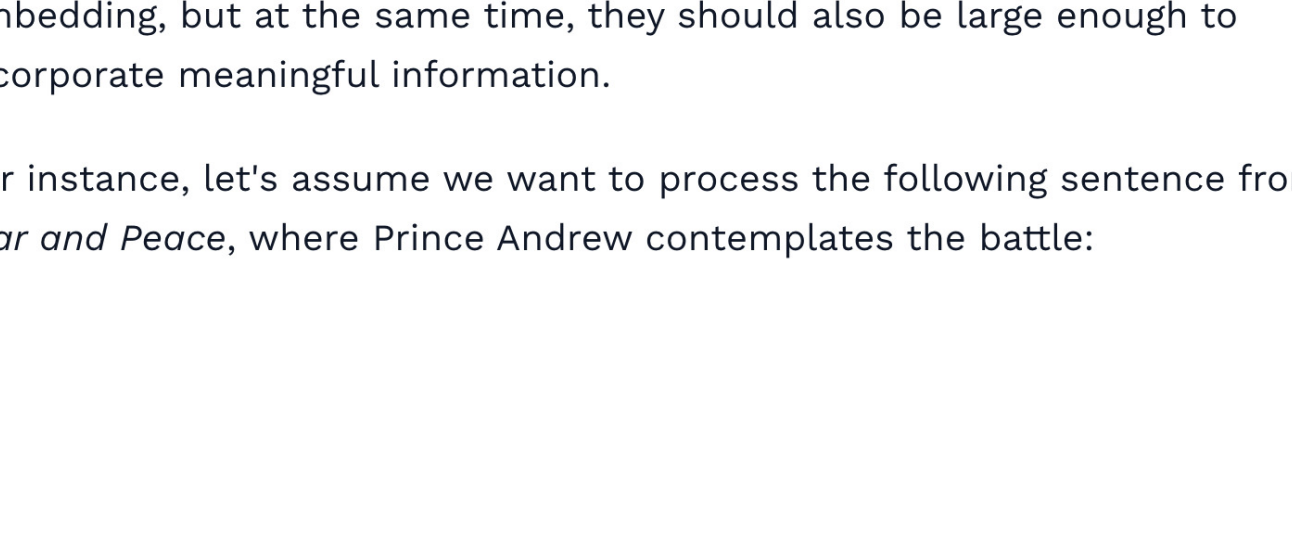


Image by author

There are several parameters to further customize the chunking process and fit it to our specific needs. A key parameter of the chunking process is the *chunk size*, which allows us to specify what the size of each chunk will be (in characters or in tokens). The trick here is that the chunks we create have to be small enough to be processed within the size limitations of the embedding, but at the same time, they should also be large enough to incorporate meaningful information.

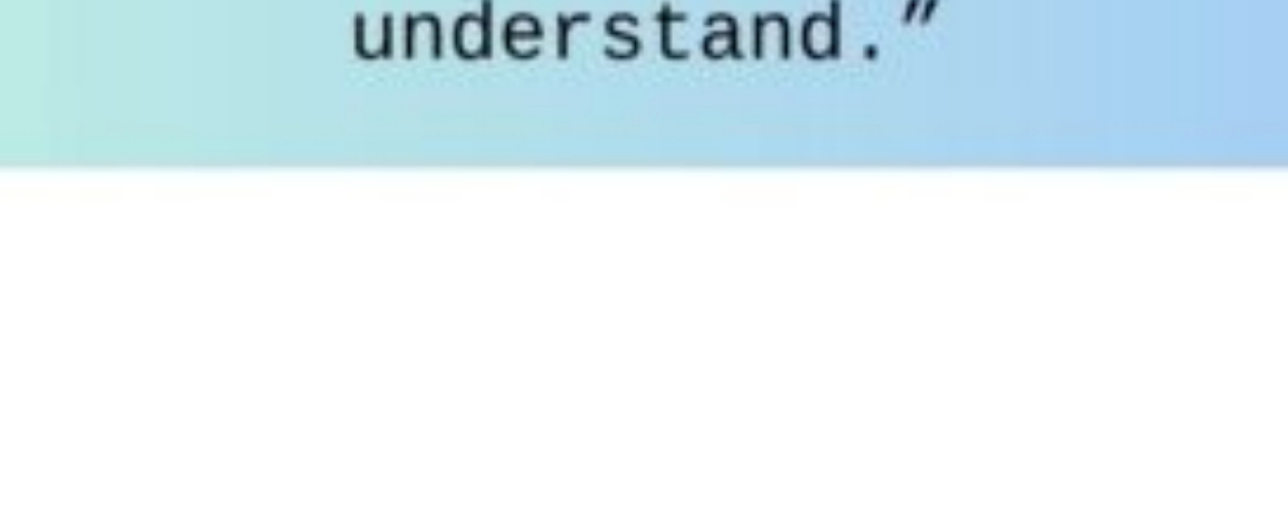
For instance, let's assume we want to process the following sentence from *War and Peace*, where Prince Andrew contemplates the battle:



the same now?" thought he.
there been here? Why was I s
as something in this life I
understand."

Image by author

Let's also assume we created the following (rather small) chunks :



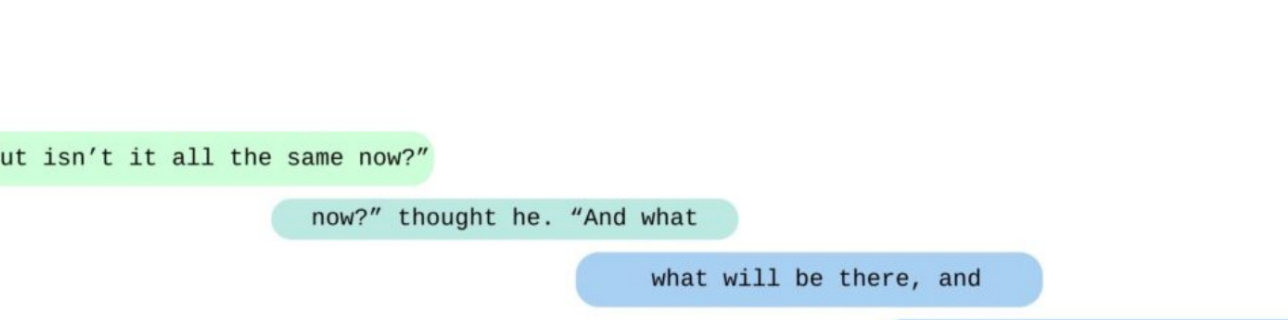
"But isn't it all the same now?" thought he.
"And what will be there, and what has there been here?
Why was I so reluctant to part with life?
There was something in this life I did not and do not understand."

Image by author

Then, if we were to ask something like *"What does Prince Andrew mean by 'all the same now'?"*, we may not get a good answer because the chunk *"But isn't it all the same now?" thought he.* does not contain any context and is vague. In contrast, the meaning is scattered across multiple chunks. Thus, even though it is similar to the question we ask and may be retrieved, it does not contain any meaning to produce a relevant response. Therefore, selecting the appropriate chunk size for the chunking process in line with the type of documents we use for the RAG, can largely influence the quality of the responses we'll be getting. In general, the content of a chunk should make sense for a human reading it without any other information, in order to also be able to make sense for the model. Ultimately, a trade-off for the chunk size exists — chunks need to be small enough to meet the embedding model's size limitations, but large enough to preserve meaning.

• • •

Another significant parameter is the chunk overlap. That is how much overlap we want the chunks to have with one another. For instance, in the *War and Peace* example, we would get something like the following chunks if we chose a chunk overlap of 5 characters.



"But isn't it all the same now?"
now?" thought he. "And what
what will be there, and
, and what has there been

Image by author

This is also a very important decision we have to make because:

- Larger overlap means more calls and tokens spent on embedding creation, which means more expensive + slower
- Smaller overlap means a higher chance of losing relevant information between the chunk boundaries

Choosing the correct chunk overlap largely depends on the type of text we want to process. For example, a recipe book where the language is simple and straightforward most probably won't require an exotic chunking methodology. On the flip side, a classic literature book like *War and Peace*, where language is very complex and meaning is interconnected throughout different paragraphs and sections, will most probably require a more thoughtful approach to chunking in order for the RAG to produce meaningful results.

• • •

But what if all we need is a simpler RAG that looks up to a couple of documents that fit the size limitations of whatever embeddings model we use in just one chunk? Do we still need the chunking step, or can we just directly make one single embedding for the entire text? The short answer is that it is always better to perform the chunking step, even for a knowledge base that does fit the size limits. That is because, as it turns out, when dealing with large documents, we face the problem of getting lost in the middle — missing relevant information that is incorporated in large documents and respective large embeddings.

## What are those mysterious 'size limitations'?

In general, a request to an embedding model can include one or more chunks of text. There are several different kinds of limitations we have to consider relatively to the size of the text we need to create embeddings for and its processing. Each of those different types of limits takes different values depending on the embedding model we use. More specifically, these are:

- **Chunk Size**, or also maximum tokens per input, or context window. This is the maximum size in tokens for each chunk. For instance, for OpenAI's `text-embedding-3-small` embedding model, the chunk size limit is 8,191 tokens. If we provide a chunk that is larger than the chunk size limit, in most cases, it will be silently truncated‼️ (an embedding is going to be created, but only for the first part that meets the chunk size limit), without producing any error.
- **Number of Chunks per Request**, or also number of inputs. There is also a limit on the number of chunks that can be included in each request. For instance, all OpenAI's embedding models have a limit of 2,048 inputs — that is, a maximum of 2,048 chunks per request.
- **Total Tokens per Request:** There is also a limitation on the total number of tokens of all chunks in a request. For all OpenAI's models, the total maximum number of tokens across all chunks in a single request is 300,000 tokens.

So, what happens if our documents are more than 300,000 tokens? As you may have imagined, the answer is that we make multiple consecutive/parallel requests of 300,000 tokens or fewer. Many Python libraries do this automatically behind the scenes. For example, LangChain's `OpenAIEmbeddings` that I use in my previous post, automatically batches the documents we provide into batches under 300,000 tokens, given that the documents are already provided in chunks.

## Reading larger files into the RAG pipeline

Let's take a look at how all these play out in a simple Python example, using the *War and Peace* text as a document to retrieve in the RAG. The data I'm using — Leo Tolstoy's *War and Peace* text — is licensed as Public Domain and can be found in Project Gutenberg.

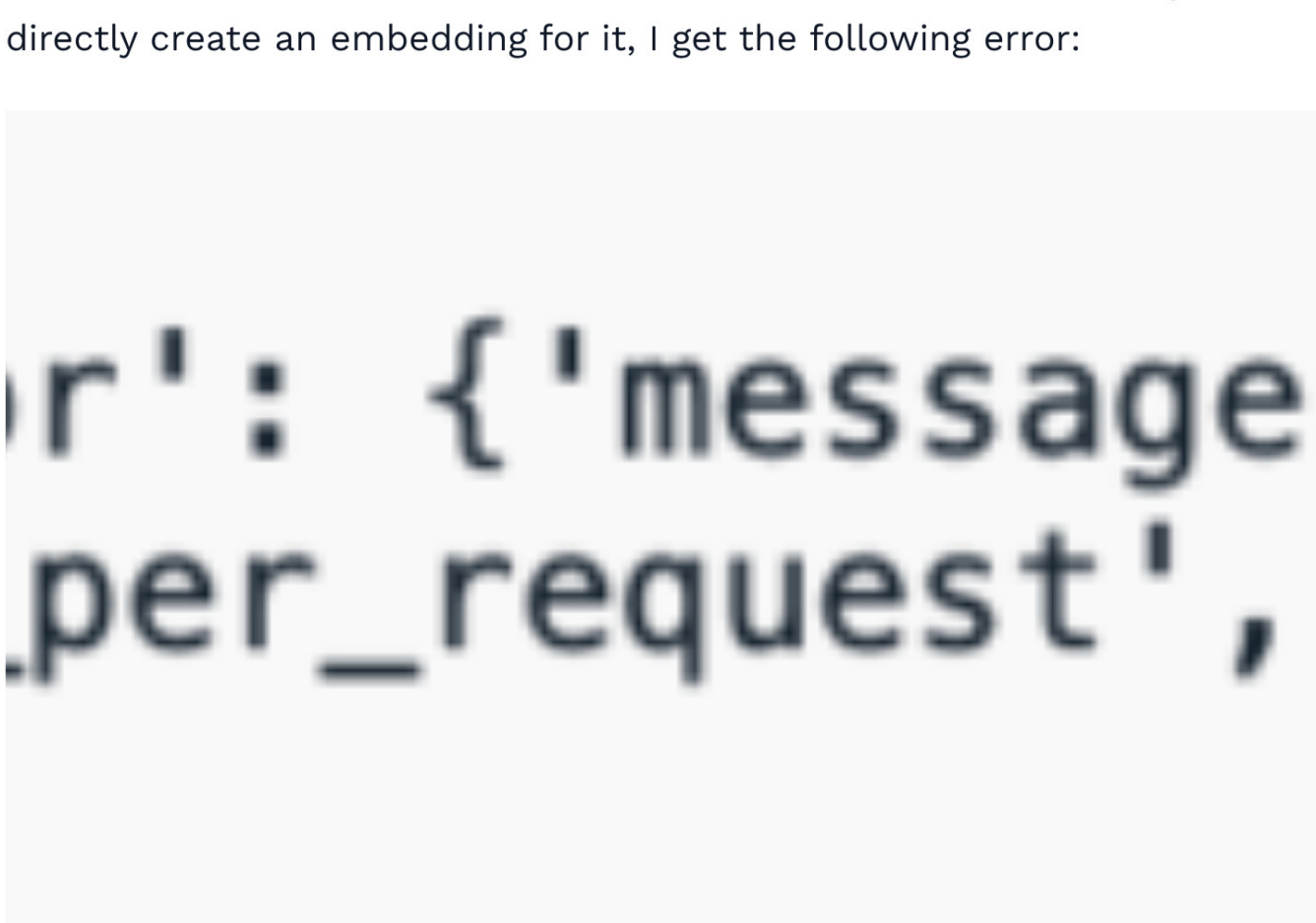
So, first of all, let's try to read from the *War and Peace* text without any setup for chunking. For this tutorial, you'll need to have installed the `langchain`, `openai`, and `faiss` Python libraries. We can easily install the required packages as follows:

```bash
pip install openai langchain langchain-community langchain-openai faiss-cpu
```

After making sure the required libraries are installed, our code for a very simple RAG looks like this and works fine for a small and simple .txt file in the `text_folder`.

```python
from openai import OpenAI

# Chat_GPT API key
api_key = "your key"

# initialize LLM
llm = ChatOpenAI(openai_api_key=api_key, model="gpt-4o-mini", temperature=0.3)

# initialize embeddings model
embeddings = OpenAIEmbeddings(openai_api_key=api_key)

# loading documents to be used for RAG
text_folder = "RAG files"

documents = []
for filename in os.listdir(text_folder):
    if filename.lower().endswith(".txt"):
        file_path = os.path.join(text_folder, filename)
        loader = TextLoader(file_path)
        documents.extend(loader.load())

# create vector database w FAISS
vector_store = FAISS.from_documents(documents, embeddings)
retriever = vector_store.as_retriever()

def main():
    print("Welcome to the RAG Assistant. Type 'exit' to quit.\n")

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            print("Exiting…")
            break

        # get relevant documents
        relevant_docs = retriever.invoke(user_input)
        retrieved_context = "\n\n".join([doc.page_content for doc in relevant_docs])

        # system prompt
        system_prompt = (
            "You are a helpful assistant. "
            "Use ONLY the following knowledge base context to answer the user. "
            "If the answer is not in the context, say you don't know.\n\n"
            f"Context:\n{retrieved_context}"
        )

        # messages for LLM
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_input}
        ]

        # generate response
        response = llm.invoke(messages)
        assistant_message = response.content.strip()
        print(f"\nAssistant: {assistant_message}\n")

if __name__ == "__main__":
    main()
```

But, if I add the *War and Peace* .txt file in the same folder, and try to directly create an embedding for it, I get the following error:



```
r': {'message'
per_request',
```

Image by author

ughh 🙃

So what happens here? LangChain's `OpenAIEmbeddings` cannot split the text into separate, less than 300,000 token iterations, because we did not provide it in chunks. It does not split the chunk, which is 777,181 tokens, leading to a request that exceeds the 300,000 tokens maximum per request.

• • •

Now, let's try to set up the chunking process to create multiple embeddings from this large file. To do this, I will be using the `text_splitter` library provided by LangChain, and more specifically, the `RecursiveCharacterTextSplitter`. In `RecursiveCharacterTextSplitter`, the chunk size and chunk overlap parameters are specified as a number of characters, but other splitters like `TokenTextSplitter` or `OpenAITokenSplitter` also allow to set up these parameters as a number of tokens.

So, we can set up an instance of the text splitter as below:

```python
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
```

… and then use it to split our initial document into chunks…

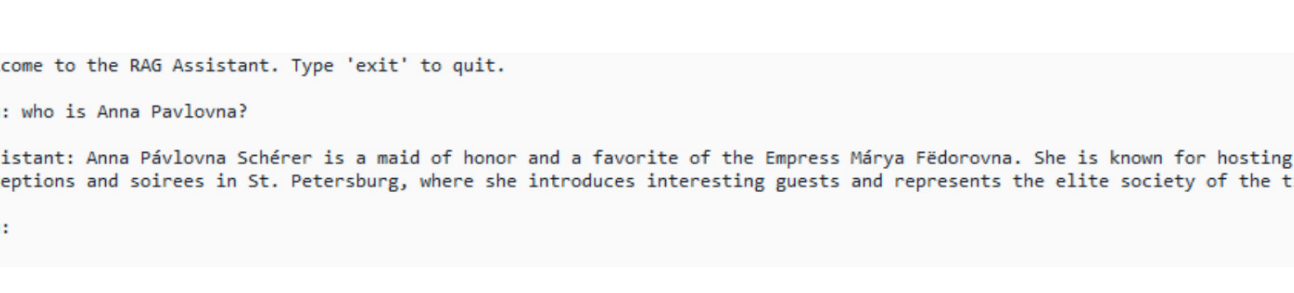
```python
split_docs = []
for doc in documents:
    chunks = splitter.split_text(doc.page_content)
    for chunk in chunks:
        split_docs.append(Document(page_content=chunk))
```

…and then use those chunks to create the embeddings…

```python
documents = split_docs

# create embeddings + FAISS index
vector_store = FAISS.from_documents(documents, embeddings)
retriever = vector_store.as_retriever()

.....
```

… and voila 🌟

Now our code can effectively parse the provided document, even if it is a bit larger, and provide relevant responses.



```
Welcome to the RAG Assistant. Type 'exit' to quit.

You: who is Anna Pavlovna?

Assistant: Anna Pávlovna Schérer is a maid of honor and a favorite of the Empress Márya Fëdorovna. She is known for hosting
receptions and soirees in St. Petersburg, where she introduces interesting guests and represents the elite society of the time.

You:
```

Image by author

## On my mind

Choosing a chunking approach that fits the size and complexity of the documents we want to feed into our RAG pipeline is crucial for the quality of the responses that we'll be receiving. For sure, there are several other parameters and different chunking methodologies one needs to take into account. Nonetheless, understanding and fine-tuning chunk size and overlap is the foundation for building RAG pipelines that produce meaningful results.

✨Thank you for reading!✨

• • •

*If you made it this far,* ***you might find pialgorithms useful*** *— a platform we've been building that helps teams securely manage organizational knowledge in one place.*

• • •

*Loved this post? Let's be friends! Join me on:*

📰***Substack*** 💌 ***Medium*** 💼***LinkedIn*** ☕***Buy me a coffee!***

WRITTEN BY
**Maria Mouschoutzi**
💖 all things data, AI/ML, tech 🙂 30 @Athens, GR

See all from Maria Mouschoutzi

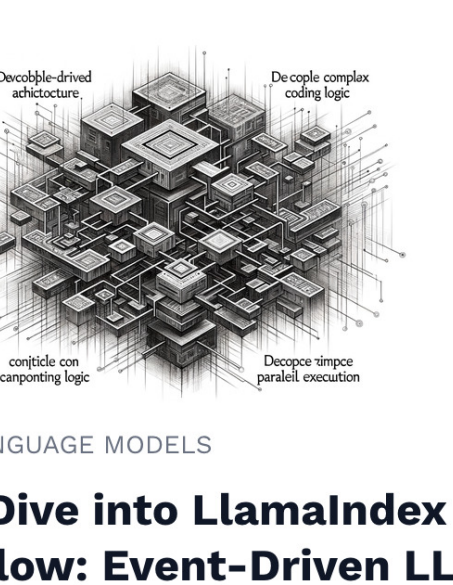
Editor's Picks Artificial Intelligence ChatGPT Data Science Python

**Share This Article**

**Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.**

Write for TDS

## Related Articles


Decobble-drived achictocture
De:cople complax coding logic
conjticle con canponting logic
Decopce timpce paralell execution

LARGE LANGUAGE MODELS

**Deep Dive into LlamaIndex Workflow: Event-Driven LLM Architecture**

What I think about the progress and shortcomings after practice

Peng Qian
December 17, 2024 • 15 min read

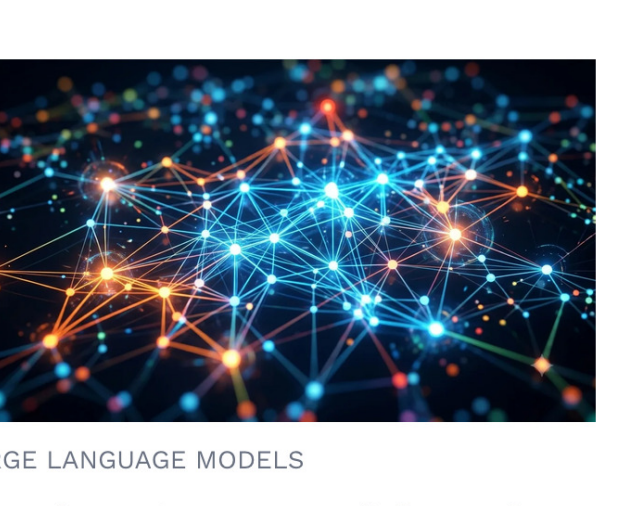
Retrieval-Augmented Generation
VS
Long-Context Large Language Model

ARTIFICIAL INTELLIGENCE

**Why Retrieval-Augmented Generation Is Still Relevant in the Era of Long-Context Language Models**

In this article we will explore why 128K tokens and more models can't fully replace using RAG.

Jérôme DIAZ
December 12, 2024 • 7 min read



LARGE LANGUAGE MODELS

**GraphRAG: A Practitioner's Guide to 6 Advanced Architectural Patterns**

Beyond basic graph retrieval: six production-oriented architectures for combining semantic search, knowledge graphs, and LLM reasoning.

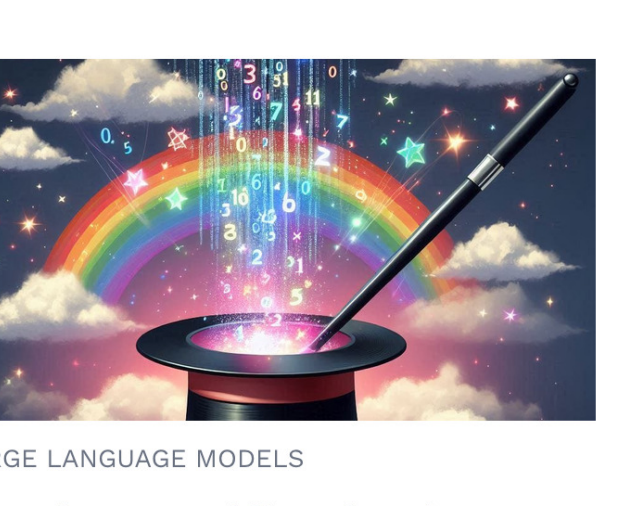
Partha Sarkar
September 20, 2026 • 20 min read

ARTIFICIAL INTELLIGENCE

**Fine-Tune Your Own Llama 2 Model in a Colab Notebook**

A practical introduction to LLM fine-tuning

Maxime Labonne
July 25, 2023 • 12 min read

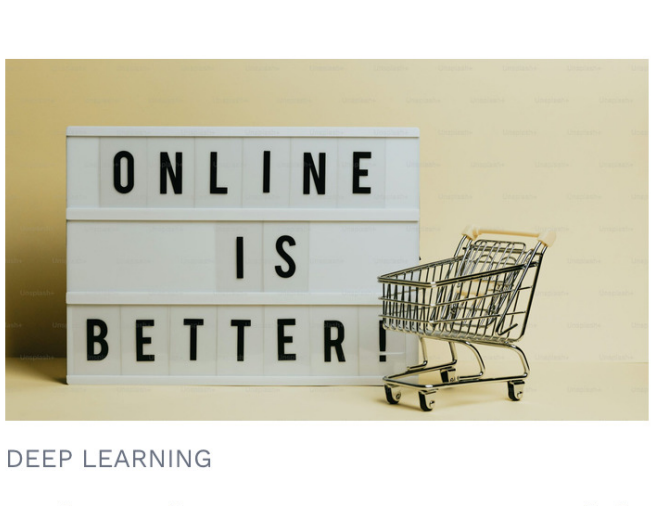


LARGE LANGUAGE MODELS

**I Took a Certification in AI. Here's What It Taught Me About Prompt Engineering.**

A software developer learns how large language models are more than just magic.

Kory Becker
June 22, 2024 • 13 min read

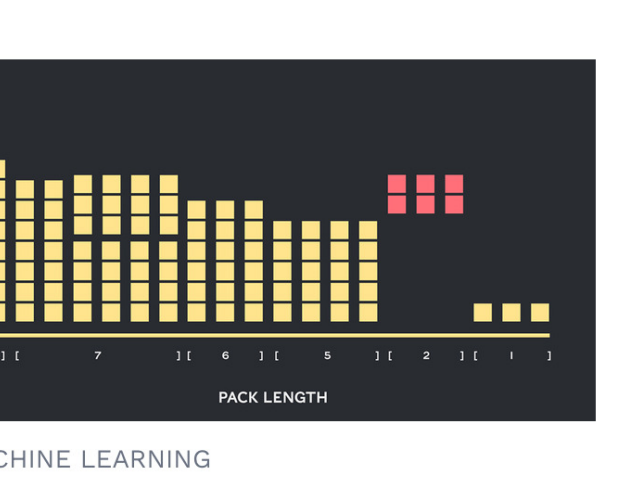


DEEP LEARNING

**Enhancing E-Commerce with Generative AI - Part 1**

From Data to Product Recommendations

Mina Ghashami
July 31, 2024 • 17 min read

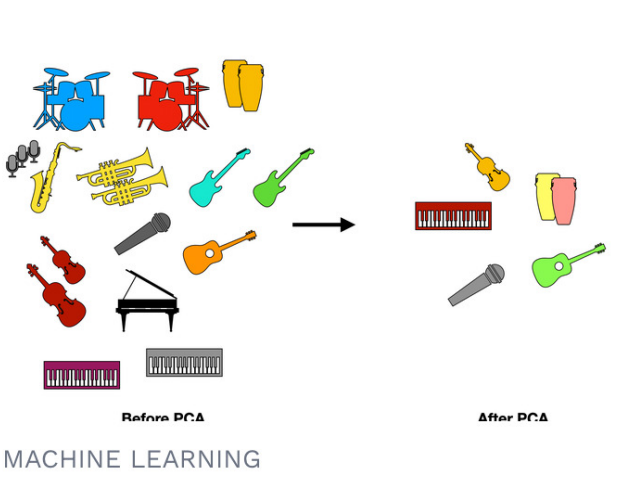


MACHINE LEARNING

**Introducing Packed BERT for 2x Training Speed-up in Natural Language Processing**

New BERT packing algorithm for more efficient training

Dr. Mario Michael Krell
July 7, 2021 • 11 min read



MACHINE LEARNING

**Principal Component Analysis (PCA)**

Intuition, math, and stonks

Shaw Talebi
February 22, 2021 • 8 min read

LARGE LANGUAGE MODELS

**Building RAGs Without A Retrieval Model Is a Terrible Mistake**

Here are my favorite techniques - one is faster, the other is more accurate.

Thuwarakesh Murallie